Project Directory Structure (example)

```
maven-web-app/
├─ src/
│  └─ main/
│     └─ ...
├─ pom.xml
├─ Dockerfile
├─ docker-compose.yml
├─ Jenkinsfile
├─ .github/
│  └─ workflows/
│     └─ ci-cd.yml
├─ buildspec.yml
├─ appspec.yml
└─ scripts/
   ├─ stop.sh
   ├─ start.sh
   ├─ pull.sh
   └─ healthcheck.sh
```

Assumptions

- Java 17 Spring Boot Maven app producing target/maven-web-app.jar.
- App is containerized and pushed to **Amazon ECR**. Deployment target is **EC2** using **CodeDeploy**, which pulls & runs via **docker-compose**.
- Replace placeholders: ${AWS_ACCOUNT_ID}, ${AWS_REGION}, ${ECR_REPO_NAME}, ${APP_NAME}, ${ENV}.

---

1) Jenkinsfile (Declarative Pipeline)

```
pipeline {
 agent any

 environment {
```

```
  AWS_REGION     = credentials('aws-region-text') // or set a plain string like 'ap-south-1'
  AWS_ACCESS_KEY_ID     = credentials('aws-access-key-id')
  AWS_SECRET_ACCESS_KEY = credentials('aws-secret-access-key')
  ECR_ACCOUNT = "${AWS_ACCOUNT_ID ?: '123456789012'}"
  ECR_REPO    = "${ECR_REPO_NAME ?: 'maven-web-app'}"
  IMAGE_TAG   = "${env.BUILD_NUMBER}"
  IMAGE_URI   = "${ECR_ACCOUNT}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPO}:${IMAGE_TAG}"
}

options { timestamps(); ansiColor('xterm'); skipDefaultCheckout(true) }

stages {
  stage('Checkout') {
    steps {
      checkout([$class: 'GitSCM', branches: [[name: '*/main']], userRemoteConfigs: [[url: 'https://github.com/your-org/your-repo.git']]])
    }
  }

  stage('Build JAR') {
    agent { label 'jdk17' }
    steps {
      sh 'mvn -q -Dmaven.test.skip=true clean package'
    }
    post { success { archiveArtifacts artifacts: 'target/*.jar', fingerprint: true } }
  }

  stage('Docker Build & Login') {
    steps {
      sh '''
        aws --version || pip3 install --upgrade awscli
        aws ecr get-login-password --region $AWS_REGION | docker login \
          --username AWS --password-stdin $ECR_ACCOUNT.dkr.ecr.$AWS_REGION.amazonaws.com
        docker build -t $ECR_REPO:$IMAGE_TAG .
        docker tag $ECR_REPO:$IMAGE_TAG $IMAGE_URI
      '''
    }
```

```
    }

    stage('Push Image to ECR') {
      steps {
        sh 'docker push $IMAGE_URI'
      }
    }

    stage('Package Artifact for CodeDeploy') {
      steps {
        sh '''
          sed -i "s|IMAGE_URI_PLACEHOLDER|$IMAGE_URI|g" docker-compose.yml
          mkdir -p bundle
          cp appspec.yml bundle/
          cp -r scripts bundle/
          cp docker-compose.yml bundle/
          cd bundle && zip -r ../codedeploy_bundle.zip .
        '''
        archiveArtifacts artifacts: 'codedeploy_bundle.zip', fingerprint: true
      }
    }

    stage('Trigger CodeDeploy (optional)') {
      when { expression { return env.CODEDEPLOY_APP && env.CODEDEPLOY_GROUP } }
      steps {
        sh '''
          aws deploy create-deployment \
            --application-name "$CODEDEPLOY_APP" \
            --deployment-group-name "$CODEDEPLOY_GROUP" \
            --region "$AWS_REGION" \
            --s3-location bucket=$S3_BUCKET,bundleType=zip,key=$S3_KEY
        '''
      }
    }
  }
}
```

Jenkins credentials needed - aws-access-key-id, aws-secret-access-key (Kind: Secret text or Username/Password) - Optionally aws-region-text as a secret text, or hardcode the region.

---

2) GitHub Actions: .github/workflows/ci-cd.yml

```yaml
name: CI-CD

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-test-docker:
    runs-on: ubuntu-latest

    permissions:
      contents: read
      id-token: write    # for OIDC to AWS

    env:
      AWS_REGION: ap-south-1
      ECR_REPO: maven-web-app
      ACCOUNT_ID: ${{ secrets.AWS_ACCOUNT_ID }}
      IMAGE_TAG: ${{ github.run_number }}
      IMAGE_URI: ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ env.AWS_REGION }}.amazonaws.com/${{ env.ECR_REPO }}:${{
env.IMAGE_TAG }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
```

```yaml
  with:
    distribution: 'temurin'
    java-version: '17'

- name: Build with Maven
  run: mvn -q -Dmaven.test.skip=true clean package

- name: Configure AWS credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
    aws-region: ${{ env.AWS_REGION }}

- name: Login to Amazon ECR
  uses: aws-actions/amazon-ecr-login@v2

- name: Build, tag, and push image
  run: |
    docker build -t $ECR_REPO:$IMAGE_TAG .
    docker tag $ECR_REPO:$IMAGE_TAG $IMAGE_URI
    docker push $IMAGE_URI

- name: Prepare CodeDeploy bundle
  run: |
    sed -i "s|IMAGE_URI_PLACEHOLDER|$IMAGE_URI|g" docker-compose.yml
    mkdir -p bundle
    cp appspec.yml bundle/
    cp -r scripts bundle/
    cp docker-compose.yml bundle/
    cd bundle && zip -r ../codedeploy_bundle.zip .

- name: Upload artifact for CodePipeline/Release
  uses: actions/upload-artifact@v4
  with:
    name: codedeploy_bundle
    path: codedeploy_bundle.zip
```

Configure GitHub secrets: AWS_ACCOUNT_ID, AWS_ROLE_TO_ASSUME (for OIDC), or fall back to AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY if you prefer static keys with aws-actions/configure-aws-credentials.

---

3) Dockerfile

```dockerfile
# Build stage
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q -e -B -DskipTests dependency:go-offline
COPY src ./src
RUN mvn -q -e -B -DskipTests package

# Runtime stage
FROM eclipse-temurin:17-jre
WORKDIR /opt/app
COPY --from=build /app/target/*SNAPSHOT*.jar /opt/app/app.jar
# or: COPY --from=build /app/target/maven-web-app.jar /opt/app/app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/opt/app/app.jar"]
```

---

4) docker-compose.yml

```yaml
services:
  app:
    image: IMAGE_URI_PLACEHOLDER  # replaced by CI with the built image URI
    ports:
      - "80:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=${ENV:-prod}
      - JAVA_TOOL_OPTIONS=-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0
    restart: always
```

---

5) AWS CodeBuild: buildspec.yml

This buildspec is suitable when CodePipeline triggers CodeBuild. It builds & pushes image to ECR, rewrites docker-compose.yml with the new tag, and outputs a **zip** bundle for CodeDeploy.

```yaml
version: 0.2

env:
  variables:
    APP_NAME: maven-web-app
    ECR_REPO: maven-web-app
    AWS_REGION: ap-south-1
  compute-type: BUILD_GENERAL1_SMALL

phases:
  install:
    runtime-versions:
      java: corretto17
    commands:
      - echo Installing dependencies
      - curl -sSL https://get.docker.com/ | sh
      - aws --version || pip3 install --upgrade awscli
  pre_build:
    commands:
      - echo Logging into Amazon ECR...
      - ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
      - aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin
$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
      - IMAGE_TAG=$(date +%Y%m%d%H%M%S)
      - IMAGE_URI=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_REPO:$IMAGE_TAG
  build:
    commands:
      - echo Building app and Docker image
      - mvn -q -Dmaven.test.skip=true clean package
      - docker build -t $ECR_REPO:$IMAGE_TAG .
      - docker tag $ECR_REPO:$IMAGE_TAG $IMAGE_URI
      - docker push $IMAGE_URI
```

```yaml
  post_build:
    commands:
      - echo Preparing CodeDeploy bundle
      - sed -i "s|IMAGE_URI_PLACEHOLDER|$IMAGE_URI|g" docker-compose.yml
      - mkdir -p bundle
      - cp appspec.yml bundle/
      - cp -r scripts bundle/
      - cp docker-compose.yml bundle/
      - cd bundle && zip -r ../codedeploy_bundle.zip .
artifacts:
  files:
    - codedeploy_bundle.zip
```

---

6) CodeDeploy appspec.yml (EC2 + Docker Compose)

```yaml
version: 0.0
os: linux
files:
  - source: /docker-compose.yml
    destination: /opt/${APP_NAME:-maven-web-app}
  - source: /scripts
    destination: /opt/${APP_NAME:-maven-web-app}/scripts
permissions:
  - object: /opt/${APP_NAME:-maven-web-app}/scripts
    mode: 755
    type: directory
hooks:
  BeforeInstall:
    - location: scripts/stop.sh
      timeout: 300
      runas: root
  AfterInstall:
    - location: scripts/pull.sh
      timeout: 300
      runas: root
```

```yaml
ApplicationStart:
  - location: scripts/start.sh
    timeout: 300
    runas: root
ValidateService:
  - location: scripts/healthcheck.sh
    timeout: 300
    runas: root
```

---

7) Deployment hook scripts (scripts/)

scripts/stop.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="/opt/${APP_NAME:-maven-web-app}"
cd "$APP_DIR"
if command -v docker-compose >/dev/null 2>&1; then
  docker-compose down || true
else
  docker compose down || true
fi
```

scripts/pull.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="/opt/${APP_NAME:-maven-web-app}"
cd "$APP_DIR"
if command -v docker-compose >/dev/null 2>&1; then
  docker-compose pull
else
  docker compose pull
fi
```

scripts/start.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="/opt/${APP_NAME:-maven-web-app}"
cd "$APP_DIR"
if command -v docker-compose >/dev/null 2>&1; then
  docker-compose up -d --remove-orphans
else
  docker compose up -d --remove-orphans
fi
```

scripts/healthcheck.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
curl -fsS http://localhost/actuator/health || curl -fsS http://localhost:8080/actuator/health || exit 1
```

---

8) Optional: pom.xml snippet (Spring Boot plugin)

```xml
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <version>${spring-boot.version}</version>
      <executions>
        <execution>
          <goals>
            <goal>repackage</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

How to adapt to your uploaded project

- **Path to JAR / build output**: adjust the COPY in Dockerfile and mvn cmd if your artifact name differs.
- **Port mapping**: update docker-compose.yml ports if your app uses a different port.
- **Non-Java projects**: swap build step for npm ci && npm run build (Node) or pip install -r requirements.txt (Python) and alter base images accordingly.
- **ECS instead of EC2**: replace appspec.yml with ECS-style AppSpec and have the build write imagedefinitions.json.

If you paste your actual directory tree and build tool (Maven/Gradle/Node/etc.), I'll tailor these files exactly to your project names and ports.